Day 3 - OOPs Polymorphism

ITSRUNTYM

1. What is Polymorphism?

Definition:
Polymorphism means "many forms". In Java (and other OOP languages), polymorphism allows the same interface or method to behave differently based on the object or context.

Key Idea:
Different classes can be treated through the same interface, and behavior will vary based on actual object types.

2. Types of Polymorphism in Java

Type	Name	Happens At	Example
Compile-Time Polymorphism	Method Overloading	Compile Time	Same method name, different params
Run-Time Polymorphism	Method Overriding	Run Time	Subclass changes parent method

Step-by-Step: What Compiles vs What Runs

1. Compilation Phase (Done by javac – Java Compiler)
Converts .java source code into .class bytecode.

What happens here?
- **Syntax checking**: Ensures code is valid (e.g., no undeclared variables, no mismatched method signatures).
- **Type checking**: Ensures method calls and overloaded methods match correctly.

- **Overloading resolution**: The compiler decides **which overloaded method** to call based on parameter types.
- **Bytecode generation**: Creates platform-independent .class files.

2. Runtime Phase (Handled by JVM)
Loads and executes the .class bytecode.

What happens here?
- **Class loading**: JVM loads necessary class files into memory.
- **Object creation**: JVM allocates memory for objects.
- **Method dispatch**: JVM decides **which overridden method** to invoke using **dynamic dispatch**.

What is Method Dispatch in Java?

Method dispatch refers to the process by which Java determines which method implementation to call when a method is invoked. This decision can happen at either compile time or runtime, depending on whether you're dealing with:

- **Overloaded methods → Static Method Dispatch**
- **Overridden methods → Dynamic Method Dispatch**

So, who does what?

Task	Done By	Compile Time / Runtime
Check method overloading validity	javac	Compile Time
Decide which overloaded method to call	javac	Compile Time
Check syntax errors	javac	Compile Time
Load classes	JVM	Runtime
Create objects	JVM	Runtime
Decide which overridden method to call	JVM	Runtime
Execute program logic	JVM	Runtime

Code Example to Visualize

```
class Parent {
   void show() {
      System.out.println("Parent show");
   }
}
class Child extends Parent {
   void show() {
      System.out.println("Child show");
   }
}
public class Test {
   public static void main(String[] args) {
      Parent p = new Child();  // Compile-time: type is Parent
      p.show();               // Runtime: JVM sees object is Child
   }
}
```

Compile Time:

- javac checks:
  - Does class Parent have a method show()? Yes → OK to compile.
  - It **does NOT check** the actual object type (Child) at this point.

Runtime:

- JVM checks:
  - p holds a Child object → call Child's overridden show() method.

---

3. Compile-Time Polymorphism (Method Overloading)

Definition:

Method Overloading means multiple methods with the same name but different parameter list in the same class.

Rules:

- Method name must be same

- Method signature must differ (number/type/order of parameters)
- Can overload by:
  - Changing number of parameters
  - Changing data types
  - Changing order of parameters (if types are different)

Code Example: Method Overloading

```
class Calculator {
   // Sum of 2 integers
   int sum(int a, int b) {
      return a + b;
   }
// Sum of 3 integers
   int sum(int a, int b, int c) {
      return a + b + c;
   }
// Sum of 2 doubles
   double sum(double a, double b) {
      return a + b;
   }
}
public class Test {
   public static void main(String[] args) {
      Calculator calc = new Calculator();
      System.out.println(calc.sum(2, 3));        // 5
      System.out.println(calc.sum(2, 3, 4));      // 9
      System.out.println(calc.sum(2.5, 3.7));     // 6.2
   }
}
```

Why Method Overloading is Compile-Time Polymorphism?

What is happening?

When you call an overloaded method (same method name, different parameters), the compiler decides which version to call based on:

- Number of parameters
- Type of parameters
- Order of parameters

So, method binding happens at compile-time.

Example:

```
class Printer {
   void print(String s) {
      System.out.println("Printing string: " + s);
   }
void print(int i) {
      System.out.println("Printing integer: " + i);
   }
}
public class Test {
   public static void main(String[] args) {
      Printer p = new Printer();
      p.print("Hello");   // Resolved to print(String) at compile time
      p.print(123);      // Resolved to print(int) at compile time
   }
}
```

Why Compile-Time?

- The **JVM does not decide** at runtime which method to call.
- The **compiler already knows** the argument types and resolves the correct method version.
- If you pass incorrect types, the compiler throws an error (not JVM).

Benefits:

- Increases readability
- Flexible method design
- Keeps related behavior in one place
- Better code organization

4. Runtime Polymorphism (Method Overriding)

Definition:

Method Overriding means subclass provides a specific implementation of a method that is already defined in its superclass.

Rules:

- Must have **same method name and signature**
- Must be **inherited** from superclass
- Only **non-static**, **non-final** methods can be overridden
- Access modifier can't be more restrictive

Code Example: Method Overriding

```
class Animal {
   void sound() {
      System.out.println("Animal makes a sound");
   }
}
class Dog extends Animal {
   void sound() {
      System.out.println("Dog barks");
   }
}
class Cat extends Animal {
   void sound() {
      System.out.println("Cat meows");
   }
}
public class Test {
   public static void main(String[] args) {
      Animal a1 = new Dog(); // Runtime polymorphism
      Animal a2 = new Cat();
a1.sound(); // Dog barks
      a2.sound(); // Cat meows
   }
}
```

Why Method Overriding is Runtime Polymorphism?

What is happening?

When you override a method in a subclass and call it using a parent class reference, the actual object (at runtime) decides which version to execute.

Example:

```
class Animal {
   void sound() {
      System.out.println("Animal sound");
   }
}
class Dog extends Animal {
   void sound() {
      System.out.println("Dog barks");
   }
}
public class Test {
   public static void main(String[] args) {
      Animal a = new Dog(); // Upcasting
      a.sound();  // Dog's version is called
// The method is resolved at runtime based on object type, not
reference type
   }
}
```

Why Runtime?

- a is declared as Animal, but it holds a Dog object.
- The **decision to call Dog's sound()** instead of Animal's is made by JVM **at runtime**, not at compile time.
- This is called **dynamic method dispatch**.

Benefits:

- Enables dynamic method binding
- Increases flexibility and reusability
- Supports **Open/Closed Principle** (extend behavior without modifying base)
- Common in **frameworks**, **APIs**, **plugin systems**

What is Casting in Java?

Casting is the conversion of one type to another. In object-oriented programming, casting between class types happens within an inheritance hierarchy (i.e., between parent and child).

1. What is Upcasting?

Definition:
Upcasting is the process of converting a subclass reference to a superclass reference.

```
Parent p = new Child(); // Upcasting
```

Why it's called "up"?
In class hierarchy diagrams, the Parent class is above the Child class — so going "up" means upcasting.

Example:

```
class Animal {
   void sound() {
      System.out.println("Animal makes a sound");
   }
}
class Dog extends Animal {
   void sound() {
      System.out.println("Dog barks");
   }
void fetch() {
      System.out.println("Dog fetches the ball");
   }
}

public class Test {
   public static void main(String[] args) {

      Animal a = new Dog(); // Upcasting

      a.sound();          // Dog's sound() is called

// a.fetch();   Compile-time error: method not found in Animal
   }
}
```

Key Points:
- Allowed **implicitly** (no cast needed)

- You can only access methods available in the **parent class**
- But overridden methods are resolved **at runtime** (polymorphism)

Use Case:

- You want to treat different child objects as a common type.
- Useful when using **collections**, **factory methods**, or **interfaces**.

2. What is Downcasting?

Definition:

Downcasting is the process of converting a superclass reference to a subclass reference.

```
Dog d = (Dog) a; // Downcasting (explicit cast required)
```

Why "down"?

We're going downward in the inheritance hierarchy — from parent to child.

Example (with Upcasting + Downcasting):

```
class Animal {
   void sound() {
      System.out.println("Animal makes a sound");
   }
}
class Dog extends Animal {
   void sound() {
      System.out.println("Dog barks");
   }
   void fetch() {
      System.out.println("Dog fetches the ball");
   }
}

public class Test {
   public static void main(String[] args) {
      Animal a = new Dog();  // Upcasting
```

```
        a.sound();          // Dog barks

// Downcasting
        Dog d = (Dog) a;
        d.fetch();          // Now we can call fetch()
    }
}
```

Important Note:

If the object is not actually an instance of the subclass, downcasting will throw ClassCastException at runtime.

```
Animal a = new Animal();
Dog d = (Dog) a; //  Runtime error: ClassCastException
```

Safe Downcasting using instanceof:

```
if (a instanceof Dog) {
    Dog d = (Dog) a;
    d.fetch();
} else {
    System.out.println("Not a Dog");
}
```

Complete Demo Code

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}
class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
void fetch() {
        System.out.println("Dog fetches");
    }
```

```
}
public class CastingExample {
   public static void main(String[] args) {

      // Upcasting
      Animal a = new Dog(); // implicit upcast
      a.sound();          // Dog barks (runtime polymorphism)

// Downcasting (safe way)
      if (a instanceof Dog) {
         Dog d = (Dog) a;  // explicit downcast
         d.fetch();       // Dog fetches
      } else {
         System.out.println("a is not a Dog");
      }

// Unsafe downcasting example
      Animal x = new Animal();
      Dog y = (Dog) x;
   }
}
```

8. Polymorphism with Interfaces

Example:

```
interface Vehicle {
   void start();
}
class Car implements Vehicle {
   public void start() {
      System.out.println("Car starts");
   }
}
class Bike implements Vehicle {
   public void start() {
      System.out.println("Bike starts");
   }
```

```
}

public class Test {
   public static void main(String[] args) {
      Vehicle v = new Car();  // Polymorphism
      v.start();              // Car starts
   }
}
```

Summary Table

Concept	Type	Resolved At	Example
Method Overloading	Compile Time	Compile Time	sum(int a, int b) vs sum(double a, double b)
Method Overriding	Runtime	Runtime	Dog extends Animal -> sound()
Upcasting	Conversion	Compile Time	Animal a = new Dog();
Downcasting	Conversion	Runtime	Dog d = (Dog) a;

Benefits of Using Polymorphism

- Clean and extensible code
- Enables **design principles** like OCP, LSP
- Simplifies code maintenance
- Helps in implementing **plugin-based** architectures
- Encourages use of **interfaces and abstraction**